基本設計書

Loggit

Version 3.0　　作成日：2026 年 2 月 22 日

1. プロダクト概要

1.1 プロダクト名

（仮）ActivityHub　— Loggit

1.2 背景・課題

VTuber 活動・Web 開発・筋トレなど複数の活動を掛け持ちする個人ユーザーが増えている一方、既存ツールには以下の課題がある。

- Jira や Redmine はチーム向けで個人には機能が重すぎる
- Notion や Google スプレッドシートは初期設定が面倒で継続しにくい
- 活動時間の記録・振り返りを手軽に行える個人向けツールが不足している

1.3 ターゲットユーザー

複数の活動（副業・趣味・自己投資など）を掛け持ちしているマルチ活動者。プロジェクトを自分でマネジメントしながら、活動時間も記録・振り返りたい個人。日本国内ユーザーを主な対象とする。

1.4 プロダクトの価値

- 個人に最適化された軽量プロジェクト管理（Jira の個人版）
- 活動ログの打刻・記録を手軽に実施
- タスクと活動時間の紐付けによる振り返りの可視化

2. 機能要件

2.1 機能一覧

機能カテゴリ	機能名	優先度
認証	ユーザー登録・ログイン・ログアウト	必須
プロジェクト	プロジェクトの作成・編集・削除	必須
プロジェクト	ステータス名のカスタマイズ（3 段階固定・名前変更可）	必須
タスク	タスクの作成・編集・削除・ステータス変更	必須
タスク	サブタスク管理（親・子・孫の 3 階層まで）	必須
タスク	優先度・期限設定	必須
タスク	全プロジェクト横断タスク一覧（ソート・フィルター付き）	必須
活動ログ	活動の開始・停止打刻	必須
活動ログ	活動ログの一覧（件数ベースページネーション）	必須
活動ログ	タスクへの時間配賦（1 ログ→多タスク）	必須
活動ログ	ダッシュボードでの週次・月次集計表示	必須
設定	時間トラッキング ON/OFF	必須
拡張	ガントチャート（タイムライン表示）	拡張
拡張	プロジェクト一覧のカード/リスト切替	拡張

3. 画面設計

3.1 画面一覧

- ログイン / 新規登録画面
- ダッシュボード（メイン画面）
- プロジェクト一覧画面
- プロジェクト詳細画面（タスクボード）
- タスク詳細（サイドパネル）
- 活動ログ画面
- タイムライン画面（拡張）
- 設定画面

3.2 共通レイアウト

サイドバーレイアウトを採用。サイドバーは常時表示とし、メインコンテンツ領域で各画面を切り替える。

サイドバー構成

- ロゴ / アプリ名
- ダッシュボード
- プロジェクト一覧
- タスク一覧（全プロジェクト横断）
- 活動ログ
- タイムライン（拡張）
- 設定
- ログアウト

3.3 ダッシュボード

アプリのメイン画面。以下のウィジェットを配置する。

- 活動打刻ボタン（開始 / 停止）＋タイマー表示
- 今日期限のタスク一覧
- プロジェクトごとの進捗サマリー
- 活動ログの週次・月次集計表示（切替可能）
- 直近の活動ログ履歴

3.4 プロジェクト一覧画面

プロジェクトをリスト形式で一覧表示する。

- 表示項目：プロジェクト名・進捗（完了タスク数/全タスク数）・期限・ステータス
- 右上に「新規作成」ボタンを配置

3.5 プロジェクト詳細画面（タスクボード）

ステータスごとにセクションを分けてタスクをリスト表示する。

- セクション：Todo / In Progress / Done（名前はカスタマイズ可能）
- 各タスク行に：タスク名・優先度・期限・メモアイコン・サブタスク数を表示
- 各セクション下部に「＋ タスク追加」ボタンを配置

3.6 タスク詳細（サイドパネル）

タスクをクリックすると画面右からサイドパネルでスライド表示。プロジェクト一覧を見ながら詳細を確認・編集できる。

- タスク名（インライン編集可能）
- ステータス・優先度・期限

- 説明・メモ（テキストエリア）
- サブタスク一覧（チェックボックス形式）＋追加ボタン
- 時間トラッキング記録（累計時間表示、総活動時間超過時は赤色警告表示）

3.7 タスク一覧（全プロジェクト横断）

全プロジェクトのタスクを横断的に表示する画面。

- デフォルト並び順：作成日降順（新しい順）
- ソート切替：due_date 昇順（期限順）/ priority 降順（優先度順）
- フィルター：プロジェクト・ステータス・優先度・期限

3.8 活動ログ画面

活動の打刻と履歴確認を行う画面。

- 開始 / 停止ボタン（状態に応じて切り替え）
- 記録中のタイマー表示
- 活動ログ履歴：日時・時間・メモを一覧表示（件数ベースページネーション）
- ログ停止後に配賦モーダルを表示（タスクへの時間割り振り、スキップ可能）
- 配賦モーダル：タスクを選択し分単位で時間を入力、複数タスクへの配賦が可能

3.9 設定画面

- プロフィール設定（名前・メールアドレス変更）
- パスワード変更
- 時間トラッキング ON/OFF 切替

4. データベース設計

4.1 ER 図（概要）

users → projects → tasks（parent_task_id で自己参照・最大 3 階層），project_statuses
users → activity_logs → task_time_logs → tasks
users → user_settings

4.2 テーブル定義

users（ユーザー）

カラム名	型	制約	説明
id	BIGINT	PK, AUTO_INCREMENT	ユーザー ID
name	VARCHAR(255)	NOT NULL	ユーザー名
email	VARCHAR(255)	NOT NULL, UNIQUE	メールアドレス
password	VARCHAR(255)	NOT NULL	ハッシュ化パスワード
timestamps	TIMESTAMP		created_at, updated_at

projects（プロジェクト）

カラム名	型	制約	説明
id	BIGINT	PK	プロジェクト ID
user_id	BIGINT	FK(users), INDEX	オーナーユーザー
name	VARCHAR(255)	NOT NULL	プロジェクト名
description	TEXT	NULL	説明
color	VARCHAR(7)	NULL	表示カラー（HEX）
due_date	DATE	NULL	プロジェクト期限
timestamps	TIMESTAMP		created_at, updated_at

project_statuses（ステータス定義）

カラム名	型	制約	説明
id	BIGINT	PK	ステータス ID
project_id	BIGINT	FK(projects), INDEX	対象プロジェクト
name	VARCHAR(100)	NOT NULL	ステータス名
order	INT	NOT NULL	表示順序
timestamps	TIMESTAMP		created_at, updated_at

※ プロジェクト作成時にデフォルトで「Todo / In Progress / Done」を自動生成
※ 3 件固定。API レベルで「追加不可・削除は2 件以上の場合のみ可」とバリデーション

tasks（タスク）

カラム名	型	制約	説明
id	BIGINT	PK	タスク ID
project_id	BIGINT	FK(projects), INDEX	所属プロジェクト
parent_task_id	BIGINT	FK(tasks), NULL	親タスク ID（サブタスク用）
status_id	BIGINT	FK(project_statuses), INDEX	ステータス
title	VARCHAR(255)	NOT NULL	タスク名
description	TEXT	NULL	説明・メモ
priority	TINYINT	NOT NULL, DEFAULT 1	優先度（1=low, 2=mid, 3=high）
due_date	DATE	NULL, INDEX	期限日
timestamps	TIMESTAMP		created_at, updated_at

※ user_id は削除。tasks→projects→user_id で所有者を辿る（不整合リスク排除）
※ priority はTINYINT で管理。アプリ側で1=low/2=mid/3=high にラベル変換（将来の変更コスト低減）
※ 階層は親・子・孫の3 階層まで。API レベルでdepth 制限をバリデーション

activity_logs（活動ログ）

カラム名	型	制約	説明
id	BIGINT	PK	ログ ID
user_id	BIGINT	FK(users), INDEX	ユーザー
project_id	BIGINT	FK(projects), NULL, INDEX	関連プロジェクト（任意）
started_at	DATETIME	NOT NULL, INDEX	開始日時（UTC 保存）
ended_at	DATETIME	NULL	終了日時（NULL=計測中）
note	TEXT	NULL	メモ（後から編集可）
timestamps	TIMESTAMP		created_at, updated_at

※ duration_minutes は持たない。総活動時間はstarted_at～ended_at から計算で導出
※ project_id=NULL でも task_time_logs 経由でタスクへの配賦は可能

※ タイムゾーン：DB はUTC 保存、表示時にAsia/Tokyo へ変換（日本限定想定）
※ project 削除時：project_id をSET NULL （ログ自体は保持）

task_time_logs（タスク時間配賦）

カラム名	型	制約	説明
id	BIGINT	PK	ID
task_id	BIGINT	FK(tasks), INDEX	対象タスク
activity_log_id	BIGINT	FK(activity_logs), NULL, INDEX	紐付け元ログ（NULL=手動入力）
duration_minutes	INT	NOT NULL	配賦時間（分単位、端数切り捨て）
note	TEXT	NULL	メモ
timestamps	TIMESTAMP		created_at, updated_at

※ 1 つのactivity_log を複数タスクに配賦可能（1 ログ→多タスク）
※ activity_log_id=NULL は手動入力による配賦（ログなしでも記録可能）
※ activity_log 削除時：activity_log_id をSET NULL （配賦記録は保持）
※ duration_minutes の合計が総活動時間を超えてもDB 制約は設けない。UI 側で赤色警告表示

user_settings（ユーザー設定）

カラム名	型	制約	説明
id	BIGINT	PK	ID
user_id	BIGINT	FK(users), UNIQUE	ユーザー
enable_task_time_tracking	BOOLEAN	DEFAULT true	時間トラッキング ON/OFF
timestamps	TIMESTAMP		created_at, updated_at

※ OFF の場合：activity_logs は引き続き記録。task_time_logs への配賦UI・機能のみ非表示

4.3 削除時の方針（CASCADE 設計）

削除対象	影響テーブル	挙動
projects 削除	project_statuses	CASCADE DELETE
projects 削除	tasks	CASCADE DELETE
projects 削除	task_time_logs	CASCADE DELETE（tasks 経由）

削除対象	影響テーブル	挙動
projects 削除	activity_logs.project_id	SET NULL（ログ自体は保持）
tasks 削除	task_time_logs	CASCADE DELETE
tasks 削除	サブタスク（tasks）	CASCADE DELETE
activity_logs 削除	task_time_logs.activity_log_id	SET NULL（配賦記録は保持）

※ FK 制約は DB レベルで実装（ON DELETE CASCADE / ON DELETE SET NULL）

4.4 DB 整合性・制約

制約内容	実装方針
同時計測中ログは 1 件のみ	API バリデーション：ended_at=NULL のログが存在する場合は開始を拒否
started_at < ended_at	CHECK 制約（DB）+ API バリデーション両方で担保
計測中ログへの配賦禁止	API バリデーション：ended_at=NULL のログへの配賦をエラー
サブタスク 3 階層まで	API バリデーション：depth 計算で 3 階層超の場合はエラー
ステータス 3 件固定	API バリデーション：4 件目の追加・2 件以下への削除を拒否

4.5 打刻例外系の仕様

例外ケース	仕様
停止忘れ	開始から 24 時間経過したら自動で ended_at をセット（started_at+24h）。ダッシュボードに「停止忘れがありました」通知を表示
日跨ぎ計測	そのまま継続記録を許可
重複停止	既に ended_at が存在する場合は API レベルでエラーを返す

4.6 インデックス設計

テーブル	カラム	目的
projects	user_id	ユーザーのプロジェクト一覧取得

テーブル	カラム	目的
tasks	project_id	プロジェクト内タスク取得
tasks	status_id	ステータス別タスク絞り込み
tasks	due_date	期限順ソート・今日期限抽出
activity_logs	user_id	ユーザーのログ一覧取得
activity_logs	project_id	プロジェクト別ログ集計
activity_logs	started_at	期間絞り込み・時系列ソート
task_time_logs	task_id	タスク別累計時間集計
task_time_logs	activity_log_id	ログからの配賦一覧取得

5. 技術スタック

カテゴリ	技術	備考
バックエンド	Laravel（PHP）	API モード（Blade 不使用）
フロントエンド	React	SPA 構成（拡張時は Next.js へ移行）
スタイリング	TailwindCSS	Jira ライクなダークデザイン
データベース	MySQL	
API 通信	REST API（JSON）	Laravel Sanctum で API 認証
挑戦枠	Next.js	React を Next.js へ移行

6. API 仕様

6.1 共通仕様

- 認証：Laravel Sanctum（Bearer トークン）
- レスポンス形式：JSON
- タイムゾーン：DB 保存は UTC、レスポンスは Asia/Tokyo 変換
- 週の開始曜日：日曜始まり固定
- 日跨ぎログの集計：開始日基準（23:00 開始→翌 2:00 終了は開始日にカウント）
- 同時計測中ログの担保：API バリデーションで担保。DB レベルの制約は将来検討
- 配賦先タスクの整合性：同一ユーザー所有チェックのみ実施（プロジェクト整合チェックは行わない）

6.2 エラーレスポンス形式

全エンドポイントで以下の形式に統一する。

HTTP ステータス	用途
200 OK	正常レスポンス
201 Created	リソース作成成功
400 Bad Request	バリデーションエラー
401 Unauthorized	認証エラー
403 Forbidden	権限エラー（他ユーザーのリソースへのアクセス）
404 Not Found	リソース未存在
422 Unprocessable	ビジネスルール違反（同時計測中ログあり等）
500 Server Error	サーバーエラー

エラーレスポンスボディ：
{ "message": "エラー内容の説明", "errors": { "field_name": ["バリデーション詳細"] } }

6.3 エンドポイント一覧

認証

エンドポイント	メソッド	説明
POST /api/register	POST	ユーザー登録
POST /api/login	POST	ログイン・トークン発行
POST /api/logout	POST	ログアウト・トークン破棄

プロジェクト

エンドポイント	メソッド	説明
GET /api/projects	GET	プロジェクト一覧取得
POST /api/projects	POST	プロジェクト作成
GET /api/projects/{id}	GET	プロジェクト詳細取得
PUT /api/projects/{id}	PUT	プロジェクト更新
DELETE /api/projects/{id}	DELETE	プロジェクト削除

エンドポイント	メソッド	説明
GET /api/projects/{id}/statuses	GET	ステータス一覧取得
PUT /api/projects/{id}/statuses/{sid}	PUT	ステータス名更新

タスク

エンドポイント	メソッド	説明
GET /api/tasks	GET	タスク一覧（横断）デフォルト：created_at 降順
GET /api/projects/{id}/tasks	GET	プロジェクト内タスク一覧
POST /api/projects/{id}/tasks	POST	タスク作成
GET /api/tasks/{id}	GET	タスク詳細取得
PUT /api/tasks/{id}	PUT	タスク更新
DELETE /api/tasks/{id}	DELETE	タスク削除（サブタスクも CASCADE）

活動ログ

エンドポイント	メソッド	説明
GET /api/activity-logs	GET	活動ログ一覧（ページネーション）
POST /api/activity-logs/start	POST	計測開始（同時計測中の場合は 422）
POST /api/activity-logs/{id}/stop	POST	計測停止（重複停止は 422）
PUT /api/activity-logs/{id}	PUT	ログ編集（note 等）
DELETE /api/activity-logs/{id}	DELETE	ログ削除（task_time_logs.activity_log_id SET NULL）
GET /api/activity-logs/summary	GET	週次・月次集計（?period=week/month）

タスク時間配賦

エンドポイント	メソッド	説明
GET /api/tasks/{id}/time-logs	GET	タスク別配賦一覧・累計時間取得
POST /api/tasks/{id}/time-logs	POST	時間配賦（手動入力・ログ紐付け）
DELETE /api/time-logs/{id}	DELETE	配賦削除

設定

エンドポイント	メソッド	説明
GET /api/settings	GET	設定取得
PUT /api/settings	PUT	設定更新（トラッキング ON/OFF 等）
PUT /api/user	PUT	プロフィール更新
PUT /api/user/password	PUT	パスワード変更

7. 非機能要件

7.1 デザイン

- Jira ライクなダークベースのクールなデザイン
- サイドバーナビゲーションを採用
- レスポンシブ対応（PC 優先）

7.2 セキュリティ

- ユーザーごとにデータが完全に分離されること
- API 認証に Laravel Sanctum を使用
- パスワードはハッシュ化して保存
- 他ユーザーのリソースへのアクセスは 403 を返す

7.3 パフォーマンス

- 個人向けのためデータ量は限定的。インデックス設計で対応
- 大量データ時は必要に応じてキャッシュ導入を検討（拡張）
- 活動ログ・タスク一覧は件数ベースページネーション（1 ページ 20 件）で対応

7.4 タイムゾーン

- DB は UTC で保存
- API レスポンスは Asia/Tokyo に変換して返す
- 日本国内ユーザー限定想定のためタイムゾーン設定は固定

8. 推奨開発順序

フェーズ	内容	詳細
1	環境構築	Laravel / React / MySQL セットアップ
2	認証機能	ユーザー登録・ログイン・ログアウト（Sanctum）
3	プロジェクト管理	CRUD + ステータスカスタマイズ
4	タスク管理	CRUD + サブタスク（3 階層） + 優先度・期限
5	活動ログ	打刻・停止・例外系・履歴表示
6	時間配賦	配賦モーダル・累計表示・超過警告
7	ダッシュボード	各ウィジェット組み合わせ・週次月次集計
8	設定・仕上げ	設定画面・UI 調整・デザイン仕上げ
拡張	ガントチャート等	余裕があれば追加実装